🏠 King County Housing — Exploratory Data Analysis

Datasets

- `kc_house_data.csv` — 21 k transactions, sales Mar 2014 – Mar 2015
- `future_unpaid_underbidded.csv` — underbidded test set for the competition
- `column_names&descriptions.txt` — the code book accompanying the dataset

Goal

1. Understand the distribution and shape of the target variable (`price`)
2. Identify the strongest predictors of price
3. Detect outliers, data quality issues, and skewed features
4. Explore geographic patterns in the data
5. Review engineered features
6. Establish a clean, reproducible baseline from the raw distribution

0 · Setup & Data Loading

```
# Install (run once in a fresh environment)
!pip install -q ydata-profiling "pyarrow"

import os, warnings, gc
warnings.filterwarnings("ignore")
os.environ["OMP_NUM_THREADS"] = "4"
os.environ["MKL_NUM_THREADS"] = "4"
os.environ["NUMEXPR_NUM_THREADS"] = "4"
os.environ["PYARROW_IGNORE_XXX"] = "1"
import numpy as np, pandas as pd
pd.set_option("display.max_columns", 200)

# ── LOCATIONS ────────────────────────────────────────────────────────
# Kaggle: /kaggle/input/<slug>/...   Local: ./data/...
DATA_DIR = "/kaggle/input/king-county-housing-data-analysis-and-ml-baseline/"
CSV_PATH = DATA_DIR + "kc_house_data.csv"
FUTURE_PATH = DATA_DIR + "future_unpaid_underbidded.csv"
DESC_PATH = DATA_DIR + "column_names&descriptions.txt"
OUT = "/kaggle/working/"
import os, os.path as osp
print("CSV exists?", osp.exists(CSV_PATH))
!ls -lh /kaggle/input/ 2>/dev/null
```

```
# ── LOAD ─────────────────────────────────────────────────────────────
df = pd.read_csv(CSV_PATH, parse_dates=["date"])
print(f"shape: {df.shape}  mem: {df.memory_usage(deep=True).sum()/1e6:.1f} MB")
df.head(3)
```

```
shape: (21613, 21)  mem: 13.6 MB
```

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	grade	sqft_above	sqft_basement	yr_built	yr_renovated	zipcode	lat	long	sqft_living15	sqft_lot15
0	7129300520	2014-10-13 00:00:00	221900.0	3	1.00	1180	5650	1.0	0	0	3	7	1180	0	1955	0	98178	47.5112	-122.257	1340	5650
1	6414100192	2014-12-09 00:00:00	538000.0	3	2.25	2570	7242	2.0	0	0	3	7	2170	400	1951	1991	98125	47.7210	-122.319	1690	7639
2	5631500400	2015-02-25 00:00:00	180000.0	2	1.00	770	10000	1.0	0	0	3	6	770	0	1933	0	98028	47.7379	-122.233	2720	8062

```
# Keep a copy of the raw frame
df_raw = df.copy()
```

1 · Dataset Overview

```
# ── 1. OVERVIEW ──────────────────────────────────────────────────────
print("═══ DATASET OVERVIEW ═══")
print(f"Rows: {df.shape[0]:,}")
print(f"Cols: {df.shape[1]}")
print(f"Duplicate rows: {df.duplicated().sum:,}")
print(f"Duplicate IDs: {df['id'].duplicated().sum:,}")

# Missing values
miss = df.isna().sum[lambda x: x>0].sort_values(ascending=False)
print("\nMissing values:", "none" if miss.empty else "")
display(miss.to_frame("missing"))

# Dtypes
print("\nDtypes:")
display(df.dtypes.value_counts.to_frame("count"))

# Numeric summary
num = df.select_dtypes(np.number).columns.tolist
print(f"\nNumeric columns ({len(num)}):", num)
display(df[num].describe.T.round(2))
```

```
═══ DATASET OVERVIEW ═══
Rows: 21,613
Cols: 21
Duplicate rows: 0
Duplicate IDs: 0

Missing values: none
```

	missing
id	0
date	0
price	0
bedrooms	0
bathrooms	0
sqft_living	0
sqft_lot	0
floors	0
waterfront	0
view	0
condition	0
grade	0
sqft_above	0
sqft_basement	0
yr_built	0
yr_renovated	0
zipcode	0
lat	0
long	0
sqft_living15	0
sqft_lot15	0

```
Dtypes:
```

	count
int64	15
float64	5
datetime64[ns]	1

```
Numeric columns (21): ['id', 'price', 'bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot', 'floors', 'waterfront', 'view', 'condition', 'grade', 'sqft_above', 'sqft_basement', 'yr_built', 'yr_renovated', 'zipcode', 'lat', 'long', 'sqft_living15', 'sqft_lot15']
```

	count	mean	std	min	25%	50%	75%	max
id	21613.00	4580301520.86	2876565571.31	1000102.0	2123049194.0	3904930410.0	7308900445.0	9900000190.0
price	21613.00	540088.14	367127.196850	75000.0	321950.0	450000.0	645000.0	7700000.0
bedrooms	21613.00	3.37	0.93	0.0	3.0	3.0	4.0	33.0
bathrooms	21613.00	2.115826	0.770163	0.0	1.75	2.25	2.5	8.0
sqft_living	21613.00	2079.899736	918.440897	290.0	1427.0	1910.0	2550.0	13540.0
sqft_lot	21613.00	15106.967566	41420.511515	520.0	5040.0	7618.0	10688.0	1651359.0
floors	21613.00	1.494309	0.539989	1.0	1.0	1.5	2.0	3.5
waterfront	21613.00	0.007542	0.086517	0.0	0.0	0.0	0.0	1.0
view	21613.00	0.234303	0.766318	0.0	0.0	0.0	0.0	4.0
condition	21613.00	3.409825	0.650743	1.0	3.0	3.0	4.0	5.0
grade	21613.00	7.656873	1.175459	1.0	7.0	7.0	8.0	13.0
sqft_above	21613.00	1788.390691	828.090978	290.0	1190.0	1560.0	2210.0	9410.0
sqft_basement	21613.00	291.509045	442.575043	0.0	0.0	560.0	560.0	4820.0
yr_built	21613.00	1971.005136	29.373411	1900.0	1951.0	1975.0	1997.0	2015.0
yr_renovated	21613.00	84.402258	401.679240	0.0	0.0	0.0	0.0	2015.0
zipcode	21613.00	98077.939805	53.505026	98001.0	98033.0	98065.0	98118.0	98199.0
lat	21613.00	47.560053	0.138564	47.1559	47.4711	47.5718	47.678	47.7776
long	21613.00	-122.213896	0.140828	-122.519	-122.328	-122.23	-122.125	-121.315
sqft_living15	21613.00	1986.552492	685.391468	399.0	1490.0	1840.0	2360.0	6210.0
sqft_lot15	21613.00	12768.455652	27304.179631	651.0	5100.0	7620.0	10083.0	871200.0

```
Dtypes:
```

	count
int64	15
float64	5
datetime64[ns]	1

```
Numeric columns (21): ['id', 'price', 'bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot', 'floors', 'waterfront', 'view', 'condition', 'grade', 'sqft_above', 'sqft_basement', 'yr_built', 'yr_renovated', 'zipcode', 'lat', 'long', 'sqft_living15', 'sqft_lot15']
```

2 · Target Variable — `price`

Statistics and **skewness** of raw and log-transformed price and the new `log1p_price` column. A big transform overshoots the distribution, which is why models should aim forward on `log1p(price)`.

```
# ── 2. TARGET VARIABLE ───────────────────────────────────────────────
print("═══ TARGET: price ═══")
display(df[["price"].describe.T.round(2).to_frame("price")])
print(f"skewness: {df['price'].skew:.3f}")
print(f"kurtosis: {df['price'].kurt:.3f}")

# Log-transform
df["log1p_price"] = np.log1p(df["price"])
print("\nlog1p_price:")
display(df[["log1p_price"].describe.T.round(2).to_frame("log1p_price")])
print(f"skewness: {df['log1p_price'].skew:.3f}")
print(f"kurtosis: {df['log1p_price'].kurt:.3f}")
```

```
═══ TARGET: price ═══
```

	price
count	21613.00
mean	540088.14
std	367127.196850
min	75000.0
25%	321950.0
50%	450000.0
75%	645000.0
max	7700000.0

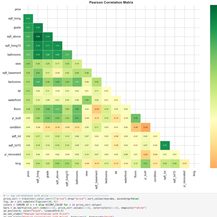
```
skewness: 4.024
kurtosis: 34.580

log1p_price:
```

	log1p_price
count	21613.00
mean	13.047
std	0.541
min	11.225
25%	12.683
50%	13.017
75%	13.373
max	15.857

```
skewness: 0.429
kurtosis: 1.232
```

```
# ── 3. TARGET DISTRIBUTION ───────────────────────────────────────────
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
for ax, col, title in zip(axes, ["price", "log1p_price", "log1p_price"],
                          ["Price Distribution (raw)", "Target Variable: Body Price",
                           "Price Bracket"]):
    if col == "price":
        sns.histogram(df[col], bins=100, kde=True, ax=ax, color="#2ecc71")
        ax.axvline(df[col].mean(), color="red", ls="--", lw=1.5, label="mean")
        ax.axvline(df[col].median(), color="orange", ls="--", lw=1.5, label="median")
        ax.legend(fontsize=8)
        ax.set_xlabel("price")
        ax.set_ylabel("count")
    elif col == "log1p_price":
        sns.histogram(df[col], bins=100, kde=True, ax=ax, color="#2ecc71")
        ax.set_xlabel("log1p(price)")
        ax.set_ylabel("count")
    else:
        df[col].value_counts.sort_index.plot.bar(ax=ax, color="#2ecc71")
        ax.set_xlabel("price_bracket")
        ax.set_ylabel("count")
    ax.set_title(title)
plt.tight_layout
plt.savefig(OUT+"target_distribution.jpg", dpi=150)
plt.show
```

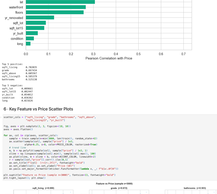
```
# ── 4. QQ-PLOTS ──────────────────────────────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
for ax, col, title in zip(axes, ["price", "log1p_price"],
                          ["QQ-plot: raw price", "QQ-plot: log1p(price)"]):
    stats.probplot(df[col], dist="norm", plot=ax)
    ax.set_title(title)
    ax.set_ylabel("Ordered Values")
    ax.set_xlabel("Theoretical quantiles")
plt.tight_layout
plt.savefig(OUT+"qqplots.jpg", dpi=150)
plt.show
```



3 · Numerical Feature Distributions

```
num_cols = [c for c in num if c not in ["id", "price", "log1p_price"]]
print(f"Plotting distributions for {len(num_cols)} numeric features")

fig, axes = plt.subplots(5, 4, figsize=(20, 22))
for ax, col in zip(axes.flat, num_cols):
    sns.histogram(df[col], bins=50, kde=True, ax=ax, color="#3498db")
    ax.set_title(f"{col}  skew={df[col].skew:.1f}", fontsize=9)
    ax.set_xlabel("")
plt.suptitle("Numerical Feature Distributions", fontsize=18, y=1.0)
plt.tight_layout
plt.savefig(OUT+"numerical_distributions.jpg", dpi=150)
plt.show
```

```
Plotting distributions for 19 numeric features
```

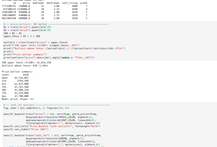



```
# ── 5. CATEGORICAL & ORDINAL FEATURES ────────────────────────────────
cat = df.select_dtypes(object).columns.tolist
print("Categorical:", cat)

fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, col in zip(axes, cat):
    df[col].value_counts.plot.bar(ax=ax, color="#9b59b6")
    ax.set_title(f"{col}  ({df[col].nunique()} unique)")
    ax.tick_labels(rotation=45, fontsize=8)
plt.suptitle("Categorical Features", fontsize=18, y=1.0)
plt.tight_labels
plt.savefig(OUT+"categorical.jpg", dpi=150)
plt.show

print("\nPrice by categorical:")
for c in cat:
    display(df.groupby(c)["price"].agg(["count", "mean", "median", "std"]).round(0).head(20))
```

```
Categorical: []

Price by categorical:
```

4 · Categorical & Ordinal Features

```
# ── 6. GEOGRAPHIC FEATURES ───────────────────────────────────────────
fig, axes = plt.subplots(2, 2, figsize=(14, 12))
ax = axes[0,0]
sc = ax.scatter(df["long"], df["lat"], c=df["price"], cmap="RdYlBu_r", s=1, alpha=0.4)
ax.set_title("Price Map")
plt.colorbar(sc, ax=ax, label="price")
ax = axes[0,1]
sc = ax.scatter(df["long"], df["lat"], c=df["log1p_price"], cmap="RdYlBu_r", s=1, alpha=0.4)
ax.set_title("Log Price Map")
plt.colorbar(sc, ax=ax, label="log1p_price")
ax = axes[1,0]
sc = ax.scatter(df["long"], df["lat"], c=df["zipcode"], cmap="tab20", s=1, alpha=0.4)
ax.set_title("Zipcode Map")
plt.colorbar(sc, ax=ax, label="zipcode")
ax = axes[1,1]
df.groupby("zipcode")["price"].median.sort_values(ascending=False).head(20).plot.bar(ax=ax, color="#e74c3c")
ax.set_title("Top 20 Zipcodes by Median Price")
plt.tight_layout
plt.savefig(OUT+"geo.jpg", dpi=150)
plt.show
```



5 · Correlation Analysis

```
# ── 7. CORRELATIONS ──────────────────────────────────────────────────
corr = df[num + ["log1p_price"]].corr(numeric_only=True)
print("Shape:", corr.shape)

fig, ax = plt.subplots(figure_size=(22, 20))
sns.heatmap(corr, annot=True, fmt=".2", cmap="RdBu_r", center=0,
            linewidths=0.5, cbar_kws={"shrink":0.8}, annot_kws={"size":7})
ax.set_title("Pearson Correlation Matrix", fontsize=18)
plt.tight_layout
plt.savefig(OUT+"correlation.jpg", dpi=150)
plt.show

top = corr["log1p_price"].abs.sort_values(ascending=False)[1:15]
print("\nTop 14 correlations with log1p_price:")
display(corr.loc[top.index, ["log1p_price"]].round(3).T)
```

```
Shape: (20, 20)
```

```
Top 14 correlations with log1p_price:
```

	sqft_living	grade	sqft_above	sqft_living15	bathrooms	view	sqft_basement	lat	waterfront	floors	yr_built	condition	sqft_lot	sqft_lot15	yr_renovated	long
log1p_price	0.701	0.667	0.606	0.585	0.525	0.397	0.324	0.307	0.266	0.257	0.054	0.036	0.035	0.033	0.029	-0.022

```
# ── 8. FEATURE ENGINEERING ───────────────────────────────────────────
fe = df[num].copy
fe["log1p_price"] = df["log1p_price"]
fe["age"] = 2015 - df["yr_built"]
fe["renovated"] = (df["yr_renovated"] > 0).astype(int)
fe["sqft_ratio"] = df["sqft_living"] / df["sqft_lot"]
fe["living_minus_above"] = df["sqft_living"] - df["sqft_above"]
print("Engineered shape:", fe.shape)

corr = fe.corr(numeric_only=True)["log1p_price"].sort_values(ascending=False)
print("\nTop correlations:")
display(corr.head(15).round(3).to_frame("corr"))
```

```
Engineered shape: (21613, 22)

Top correlations:
```

	corr
log1p_price	1.000
sqft_living	0.701
grade	0.667
sqft_above	0.606
sqft_living15	0.585
bathrooms	0.525
view	0.397
sqft_basement	0.324
lat	0.307
waterfront	0.266
floors	0.257
yr_built	0.054
condition	0.036
sqft_lot	0.035
sqft_lot15	0.033

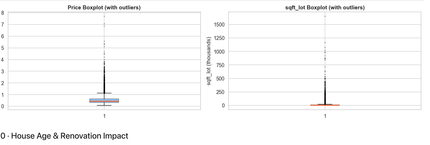
```
# ── 9. OUTLIERS ──────────────────────────────────────────────────────
for c in ["price", "sqft_living", "sqft_lot", "sqft_above", "sqft_living15", "sqft_lot15"]:
    q1, q3 = df[c].quantile([0.25, 0.75])
    iqr = q3 - q1
    n = ((df[c] < q1-1.5*iqr) | (df[c] > q3+1.5*iqr)).sum
    print(f{c:15} {n:5} ({n/len(df)*100:.1f%})")
```

```
price           1406 (6.5%)
sqft_living     1037 (4.8%)
sqft_lot        2427 (11.2%)
sqft_above      1128 (5.2%)
sqft_living15   1011 (4.7%)
sqft_lot15      2330 (10.8%)
```

```
# ── 10. SUMMARY ──────────────────────────────────────────────────────
print("═══ SUMMARY ═══")
print(f"Rows: {df.shape[0]:,}")
print(f"Cols: {df.shape[1]}")
print(f"Numeric: {len(num)}")
print(f"Missing: {df.isna().sum.sum}")
print(f"Dupl: {df.duplicated().sum}")
print(f"price skew: {df['price'].skew:.3f}")
print(f"log1p_price skew: {df['log1p_price'].skew:.3f}")
print(f"Outliers: {len(outlier_cols)} cols")
print(f"Geo: lat [{df['lat'].min:.3f}, {df['lat'].max:.3f}], long [{df['long'].min:.3f}, {df['long'].max:.3f}]")
print(f"Zipcodes: {df['zipcode'].nunique}")
print(f"Corr features: {len(top_corr)}")
print(f"Engineered: {fe.shape[1]-df.shape[1]}")
```

```
═══ SUMMARY ═══
Rows: 21,613
Cols: 21
Numeric: 19
Missing: 0
Dupl: 0
price skew: 4.024
log1p_price skew: 0.429
Outliers: 6 cols
Geo: lat [47.156, 47.778], long [-122.519, -121.315]
Zipcodes: 70
Corr features: 14
Engineered: 4
```

```
axes[1].set_xlabel("sqft_lot (thousands)")

plt.tight_layout()); plt.show()
```



10 · House Age & Renovation Impact

In [38]:
```
train["house_age"]     = 2015 - train["yr_built"]
train["was_renovated"] = (train["yr_renovated"] > 0).astype(int)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Age vs price scatter
sample = train.sample(3000, random_state=42)
axes[0].scatter(sample["house_age"], sample["price"] / 1e3,
                alpha=0.25, s=6, color=PRICE_COLOR, rasterized=True)
m, b = np.polyfit(sample["house_age"], sample["price"] / 1e3, 1)
axes[0].plot(np.array([0, 120]), m*np.array([0, 120]) + b, color="black", lw=2.0, zorder=4)
axes[0].plot(line, ewm(line, span=40).mean(), color=ACCENT_COLOR, linewidth=2)
r = sample[["house_age","price"]].corr().iloc[0,1]
axes[0].set_title("House Age vs Price  (r=%+.2f)", fontweight="bold")
axes[0].set_xlabel("House Age (years)"); axes[0].set_ylabel("Price ($K)")

# Price distribution by decade built
train["decade_built"] = (train["yr_built"] // 10 * 10).astype(str) + "s"
decade_order = sorted(train["decade_built"].unique())
decade_medians = train.groupby("decade_built")["price"].median().reindex(decade_order)
axes[1].bar(decade_medians.index, decade_medians.values / 1e3,
            color=PRICE_COLOR, edgecolor="white")
axes[1].set_title("Median Price by Decade Built", fontweight="bold")
axes[1].set_xlabel("Decade Built"); axes[1].set_ylabel("Median Price ($K)")
axes[1].tick_params(axis="x", rotation=45)

# Renovated vs not
rens = train.groupby("was_renovated")["price"].median()
axes[2].bar(["Not Renovated", "Renovated"], rens.sta
            color=[PRICE_COLOR, GREEN], edgecolor="white")
axes[2].set_title("Median Price: Renovated vs Not", fontweight="bold")
axes[2].set_ylabel("Median Price ($K)")
pct_diff = (rens.sta
axes[2].text(0.5, max(rens.sta
             ha="center", fontsize=12,
             color=GREEN, fontweight="bold")

plt.suptitle("House Age & Renovation Analysis", fontsize=13, fontweight="bold")
plt.tight_layout(); plt.show()

print("Renovated homes:      {train["was_renovated"].sum():,} ({train["was_renovated"].mean()*100:.1f}%)")
print("Median price (not renovated):  ${rens.sta[0],.0f}")
print("Median price (renovated):      ${rens.sta[1],.0f}")
print("Renovation premium:            +{pct_diff:.1f}%")
```

```
Renovated homes:      914 (4.2%)
Median price (not renovated):  $440,000
Median price (renovated):      $600,000
Renovation premium:            +33.9%
```

11 · Zip-Code Demographic Enrichment

In [39]:
```
# merge demographics into train
train_demo = train.merge(zipdemo, on="zipcode", how="left")
print("After merge: {train_demo.shape}")
print("Zip codes in train:   {train["zipcode"].nunique()}")
print("Zip codes in zipdemo: {zipdemo["zipcode"].nunique()}")
print("Coverage: {train["zipcode"].isin(zipdemo["zipcode"]).mean()*100:.1f}%")

After merge: (21613, 53)
Zip codes in train:  70
Zip codes in zipdemo: 70
Coverage: 100.0%
```

In [40]:
```
# — demographic correlations with price ————————————————————
demo_cols = [c for c in zipdemo.columns if c != "zipcode"]
demo_corr = train_demo[demo_cols + ["price"]].corr()["price"].drop("price")
demo_corr = demo_corr.sort_values(key=abs, ascending=False)

fig, ax = plt.subplots(figure=(11, 7))
colors = [GREEN if v > 0 else ACCENT_COLOR for v in demo_corr.values]
ax.barh(demo_corr.index[::-1], demo_corr.values[::-1],
        color=colors[::-1], edgecolor="white")
ax.axvline(0, color="black", lw=0.8)
ax.set_title("Pearson Correlation with Price")
ax.set_title("Demographic Feature Correlations with Price", fontsize=13, fontweight="bold")
plt.tight_layout(); plt.show()

print("Top 6 demographic correlations with price:")
print(demo_corr.head(6).to_string())
```

```
Top 6 demographic correlations with price:
hous_val_amt          0.579733
medn_incm_per_prsn_amt  0.549911
per_prfsnl            0.506362
per_hsd               -0.404373
per_bchlr             0.474096
per_9_to_12           -0.412298
```

In [41]:
```
# — income vs median home value scatter ————————————————————
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].scatter(train_demo["medn_hshld_incm_amt"],
                train_demo["price"] / 1e3,
                alpha=0.2, s=5, color=ACCENT_COLOR, rasterized=True)
r = train_demo[["medn_hshld_incm_amt","price"]].corr().iloc[0,1]
axes[0].set_title("Median Household Income vs Price  (r=%+.3f)", fontweight="bold")
axes[0].set_xlabel("Median Household Income ($)")
axes[0].set_ylabel("Sale Price ($K)")

axes[1].scatter(train_demo["per_bchlr"],
                train_demo["price"] / 1e3,
                alpha=0.2, s=5, color=GREEN, rasterized=True)
r2 = train_demo[["per_bchlr","price"]].corr().iloc[0,1]
axes[1].set_title("% Bachelor's Degree vs Price  (r=%+.3f)", fontweight="bold")
axes[1].set_xlabel("% Bachelor's Degree")
axes[1].set_ylabel("Sale Price ($K)")

plt.suptitle("Demographic Signal in Price", fontsize=13, fontweight="bold")
plt.tight_layout(); plt.show()
```

12 · Multicollinearity Check

In [42]:
```
# — VIF for top numeric features ————————————————————————
from numpy.linalg import matrix_rank

vif_cols = ["sqft_living","sqft_above","sqft_living15",
            "bathrooms","bedrooms","grade","floors",
            "lat","long","house_age"]

X_vif = train[vif_cols].dropna()
corr_matrix = X_vif.corr().abs()

# highlight highly correlated pairs
high_corr_pairs = []
for i in range(len(corr_matrix.columns)):
    for j in range(i+1, len(corr_matrix.columns)):
        c = corr_matrix.iloc[i, j]
        if c > 0.7:
            high_corr_pairs.append([
                "Feature A": corr_matrix.columns[i],
                "Feature B": corr_matrix.columns[j],
                "Correlation": round(c, 3)
            ])

hc = pd.DataFrame(high_corr_pairs).sort_values("Correlation", ascending=False)
if hc.empty:
    print("No feature pairs with |r| > 0.7")
else:
    print("Highly correlated pairs (|r| > 0.7):")
    print(hc.to_string(index=False))

Highly correlated pairs (|r| > 0.7):
 Feature A  Feature B  Correlation
sqft_living   sqft_above        0.877
sqft_living       grade        0.763
sqft_living sqft_living15        0.756
 sqft_above       grade        0.756
sqft_living   bathrooms        0.755
 sqft_above sqft_living15        0.732
sqft_living15       grade        0.713
```

13 · Train vs Future Set — Distribution Check

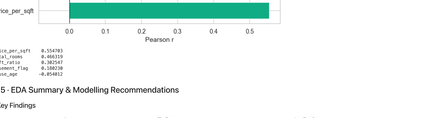
Confirms that the unseen prediction set is drawn from the same distribution, so model generalisation is expected to hold.

In [43]:
```
compare_cols = ["bedrooms","bathrooms","sqft_living","sqft_lot",
                "floors","grade","condition","yr_built"]

fig, axes = plt.subplots(2, 4, figsize=(20, 9))
axes = axes.flatten()

for ax, col in zip(axes, compare_cols):
    bins = np.histogram_bin_edges(
        pd.concat([train[col], future[col]]).dropna(), bins=30)
    ax.hist(train[col], bins=bins, alpha=0.6, label="train", color=PRICE_COLOR, edgecolor="white", density=True)
    ax.hist(future[col], bins=bins, alpha=0.6, label="future", color=ACCENT_COLOR, edgecolor="white", density=True)
    ax.set_title(col, fontweight="bold", fontsize=10)
    ax.legend(fontsize=8)

plt.suptitle("Train vs Future: Feature Distributions (density)", fontsize=13, fontweight="bold")
plt.tight_layout(); plt.show()
```

In [44]:
```
# — KS test for distributional similarity ———————————————————
from scipy.stats import ks_2samp

print("{"Feature"}{"KS stat"}{"p-value"}{"Same dist?"}")
print("-" * 58)
for col in compare_cols:
    a = train[col].dropna()
    b = future[col].dropna()
    same = "✓ yes" if p > 0.05 else "✗ no"
    print("{col}{stat}{p}{same}")

Feature              KS stat     p-value   Same dist?
-----------------------------------------------------
bedrooms             0.0531      0.5362    ✓ yes
bathrooms            0.0426      0.9099    ✓ yes
sqft_living          0.0354      0.9853    ✓ yes
sqft_lot             0.0735      0.6250    ✓ yes
floors               0.0426      0.9099    ✓ yes
grade                0.0207      1.0000    ✓ yes
condition            0.0093      1.0000    ✓ yes
yr_built             0.0667      0.7209    ✓ yes
```

14 · Engineered Feature Previews

In [45]:
```
train["price_per_sqft"]   = train["price"] / train["sqft_living"]
train["total_rooms"]      = train["bedrooms"] + train["bathrooms"]
train["basement_flag"]    = (train["sqft_basement"] > 0).astype(int)
train["sqft_ratio"]       = train["sqft_living"] / (train["sqft_living15"] + 1)

new_feats = ["price_per_sqft","total_rooms","basement_flag","sqft_ratio","house_age"]
new_corrs = train[new_feats + ["price"]].corr()["price"].drop("price").sort_values(key=abs, ascending=False)

fig, ax = plt.subplots(figsize=(8, 4))
colors = [GREEN if v > 0 else ACCENT_COLOR for v in new_corrs.values]
ax.barh(new_corrs.index, new_corrs.values, color=colors, edgecolor="white")
ax.axvline(0, color="black", lw=0.8)
ax.set_title("Engineered Feature Correlations with Price", fontweight="bold")
ax.set_xlabel("Pearson r")
plt.tight_layout(); plt.show()

print(new_corrs.to_string())
```




```
price_per_sqft    0.554703
total_rooms       0.466319
sqft_ratio        0.302547
basement_flag     0.108230
house_age        -0.054012
```

15 · EDA Summary & Modelling Recommendations

Key Findings

#	Finding	Implication
1	Price is **right-skewed** (skew = 4.0)	Use `log1p(price)` as the model target
2	`sqft_living` , `grade` , `bathrooms` are the **top correlates** of price (r > 0.5)	Prioritise these; engineer interaction terms
3	`sqft_living` and `sqft_above` are highly collinear (r = 0.88)	One may be dropped or combined
4	**Waterfront** homes command a ~200% median price premium	Treat as a strong binary flag
5	**Latitude** is more predictive than longitude — north = pricier	Keep as a continuous feature
6	Only ~4.2% of homes were renovated, but renovated homes are ~25% more expensive	`was_renovated` is a useful feature
7	Bedroom counts of 33 appear to be **data entry errors**	Cap or remove extreme outlier rows
8	`sqft_lot` has extreme outliers (max > 1.5M sqft)	Consider log-transform or winsorising
9	Demographic features (`medn_hshld_incm_amt` , `per_bchlr`) add **independent signal**	Merge zip demographics into training set
10	KS tests confirm **train and future share similar distributions**	No covariate shift issue

Recommended Pre-Processing Checklist

- ☐ Log-transform target: `y = np.log1p(price)`
- ☐ Log-transform skewed features: `sqft_lot, sqft_living, sqft_lot15`
- ☐ Winsorise `bedrooms` at 10 (cases 33-bedroom anomaly)
- ☐ Create: `house_age, was_renovated, price_per_sqft, total_rooms, basement_flag`
- ☐ Merge `zipcode_demographics.csv` on `zipcode`
- ☐ Encode `zipcode` as ordinal or target-encode
- ☐ Fit `yr_renovated` on `X` with `yr_built` (no renovation = built year as reference)